Laboratorium PSS, rok 2023.

Uwagi odnośnie stylu programowania

1. Programy mogą być pisane wyłącznie w języku C++, w stylu orientowanym obiektowo. Zaleca się stosowanie wersji Standardu 2011 i nowszych.
2. Wymagany jest podział kodu na osobne jednostki kompilacji odpowiadające poszczególnym klasom (np. obiekt dyskretny, regulator, regulator PID, regulator GPC, itd.). Tylko w szczególnych przypadkach w jednej jednostce kompilacji może być zdefiniowanych kilka klas, stanowiących spójny moduł programu.
3. Wszystkie jednostki kompilacji, które nie zawierają definicji i implementacji graficznego interfejsu użytkownika, muszą być napisane w standardzie ISO języka C++.
4. Szczególnie należy zwrócić uwagę na enkapsulację:
   - klasa nie powinna udostępniać zbyt dużej liczby publicznych funkcji składowych,
   - nie powinny – bez istotnej potrzeby – być publicznymi dane składowe klasy,
   - jeżeli pewne funkcje nie zmieniają obiektu klasy, powinny być zdeklarowane jako „const”,
   - jeżeli funkcje pobierają argumenty przez referencje, a ich nie modyfikują, argumenty powinny być być kwalifikowane przez „const”
   - przyjaźń powinna być stosowana wyłącznie w uzasadnionych przypadkach (np. na etapie debugowania, wyłączana dyrektywami preprocesora).
5. Pisany kod powinien obejmować kontrolę poprawności opartą o wyjątki – dotyczy to zwłaszcza (lecz nie tylko) kontroli wprowadzanych przez użytkownika danych czy danych odczytywanych z dysku. Dokumentacja powinna zawierać informacje na temat gwarancji bezpieczeństwa (w sensie wyjątków), z jakimi dana funkcja została napisana.
   - Niektóre funkcje (np. destruktory) muszą dawać gwarancję niewyrzucania wyjątków.
   - Inne funkcje muszą dawać silną gwarancję bezpieczeństwa (np. funkcja zmieniająca nastawy części P, I oraz D regulatora PID nie może zmienić tylko nastawy części P, a potem zgłosić wyjątku, gdyż tak nastrojony regulator może spowodować niestabilność układu).
   - Pozostałe funkcje muszą być napisane w sposób zapobiegający wyciekowi zasobów.
   - Należy wykorzystać technikę RAII aby nie dopuścić do wycieku zasobów.
6. Pisany kod musi być **na bieżąco** komentowany:
   - szczegółowo komentowany musi być cały interfejs publiczny i chroniony klasy – komentarz powinien określać co dana metoda robi, jakie parametry przyjmuje, jaką wartość zwraca, jakie może zgłaszać wyjątki,
   - wymienione wyżej komentarze należy zapisywać w plikach nagłówkowych,
   - komentowane powinny być także kluczowe fragmenty kodu programu,
   - komentarze powinny umożliwić wygenerowanie dokumentacji napisanych klas, która jest warunkiem koniecznym zaliczenia (zaleca się system *Doxygen*),
   - dokumentacja musi zawierać diagramy zależności klas zdefiniowanych w projekcie (wszystkie) i wywołań funkcji (wskazane przez prowadzącego),
   - końcowa wersja dokumentacji powinna zawierać także krótki opis interfejsu graficznego programu.
7. Należy korzystać z pojemników i algorytmów dostarczanych przez bibliotekę standardową. W szczególności **zabrania się korzystania z tablic języka C** bez precyzyjnego uzasadnienia potrzeby ich wykorzystania (w zamian należy wykorzystywać pojemniki lub array<>). Nie należy także, bez istotnej potrzeby, korzystać z tablic znakowych, w zamian stosując pojemnik „string”. Do przechowywania konfiguracji zaleca się wykorzystanie pojemnika „map”.
8. Należy korzystać z iteratorów i algorytmów standardowych. W szczególności zabrania się wykorzystywania pętli „for” do operacji na pojemnikach, takich jak kopiowanie czy mnożenie dwóch pojemników przez siebie.
9. Pożądane i premiowane będą programy **korzystające z mechanizmów obiektowych** (dziedziczenie, zawieranie klas, polimorfizm). Nie należy jednak stosować tych mechanizmów bez przemyślenia. W szczególności nie należy nadużywać dziedziczenia publicznego.
10. Pożądane i premiowane będą programy korzystające z wzorców projektowych.